Distributed Systems
Use Case: P2P

Pedro García López
pedro.garcia@urv.cat

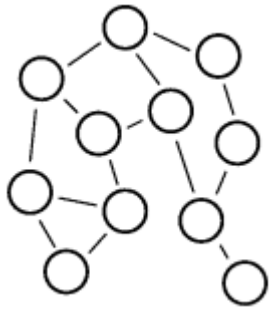
Copyright

- © University Rovira i Virgili

- Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.1 or any later version published by the Free Software Foundation; provided its original author is mentioned and the link to http://libre.act-europe.fr/ is kept at the bottom of every non-title slide. A copy of the license is available at:

  http://www.fsf.org/licenses/fdl.html

What is Peer 2 Peer ?



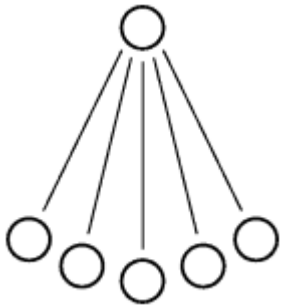
- A distributed system architecture
  - Decentralized control
  - Typically many nodes, but unreliable and heterogeneous
  - Nodes are symmetric in function
  - Take advantage of distributed, shared resources (bandwidth, CPU, storage) on peer-nodes
  - Fault-tolerant, self-organizing
  - Operate in dynamic environment, frequent join and leave is the norm

P2P Characteristics

- Decentralization
- Scalability -> avoid hotspots and bottlenecks!
- Ad-hoc connectivity
- Self-organization
- Local information
- Fault-resilience

- And also: anonimity, security, trust, replication, caching, coordination, …

What is NOT P2P ?



- A centralized system
  - Client/Server Architecture
  - Master/Slave models
  - Dumb terminals / Active broadcaster
- Problems:
  - Scalability
  - Bottleneck (hotspot)
  - Cost

Relevance of P2P

- According to several internet service providers, more than 50% of Internet traffic is due to P2P applications, sometimes even as much as 75%.
- It is a paradigm shift from coordination to **cooperation**, from centralization to **decentralization**, and from control to **incentives**

Why P2P is important (for the society)?

- Organizational shift from hierarchical / centralized models to decentralized / network models
- “The Rise of the Network Society”, Manuel Castells
- P2P can enable the construction of worldwide communities
- Example 1. Teaching: classical lecture/exam model Vs collaborative learning
- Example 2. Mass media: big and few broadcasters (TV, radio) Vs public user publishing (Web blogs, net radio, net TV)

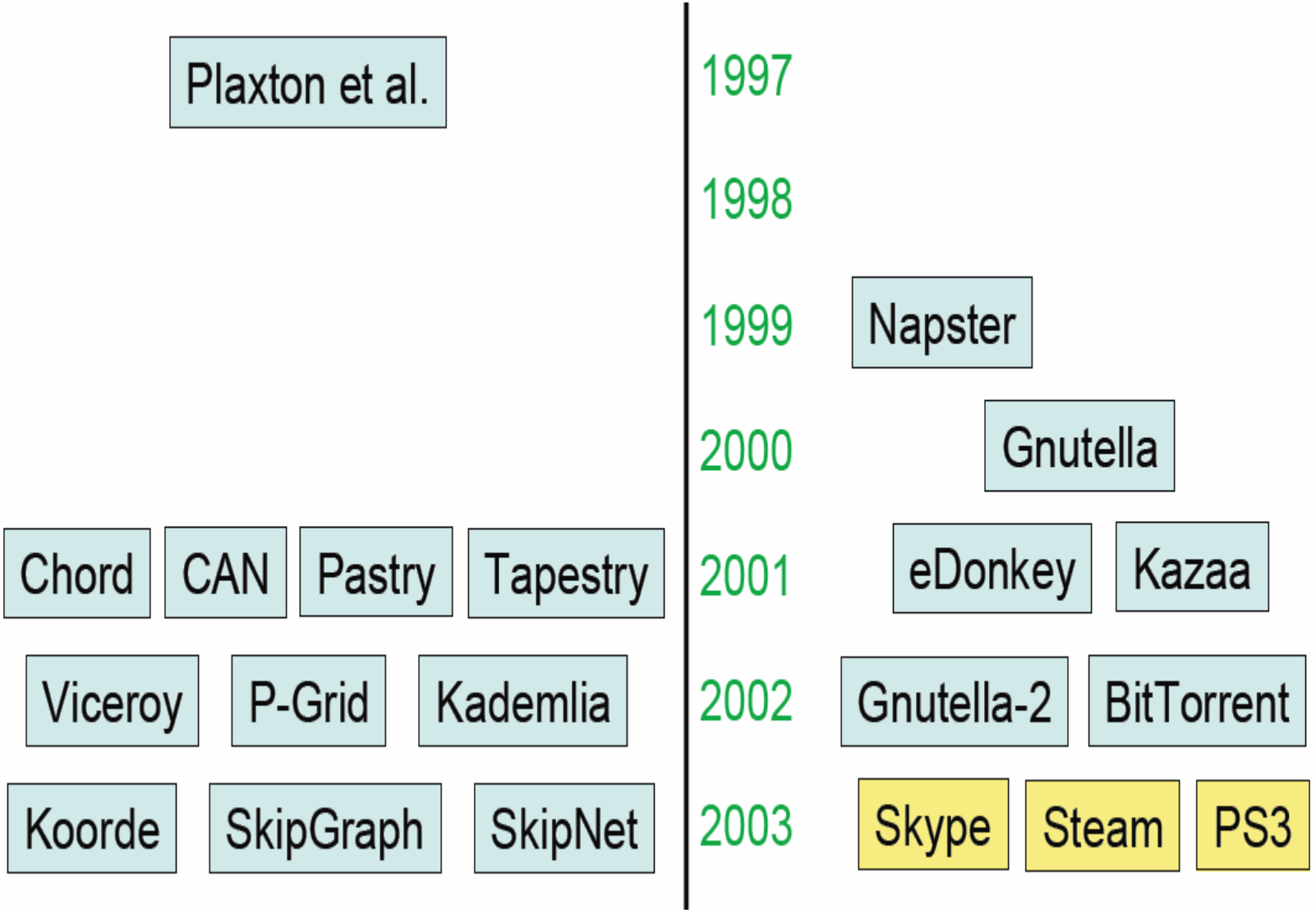
History of P2P: prehistory

- 1969- ARPANET
  - First killer applications like FTP and Telnet were Client/server
- 1979- USENET
  - UUCP, NNTP
  - Decentralized control, avoid network flood, path header, …
- DNS – 1983
  - Scalability !!!
  - Hierarchical Design
  - Distributed load
  - Caching
  - Query delegation

Internet Explosion (1995-1999)

- The switch to client/server
  - HTTP, Chat, Mail, IM
- Breakdown of cooperation
  - Spam, TCP rate equation, congestion
- Firewalls, dynamic IP, NATs
- Asymetric Bandwidth

History of P2P

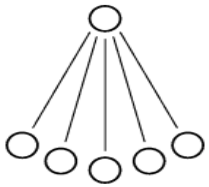


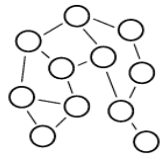
P2P Infrastructures

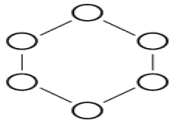
- Overlay Network:
  - A network on top of a network (IP)
  - Application level routers working in top of an IP network infrastructure
  - Redundacy ?
- Why Overlay Networks ?
  - Focused on application-level services that underlying layers do not offer
  - One-to-Many & Many-to-Many apps, IP Multicast is not supported by most Internet routers
  - IP substrate is difficult to change (IPv6 transition)

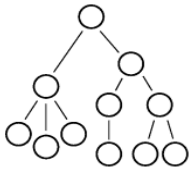
P2P Architectural models

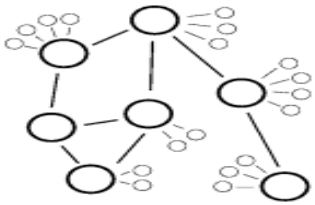
- Centralized (Napster)
- Decentralized
  - Unstructured (Gnutella)
  - Structured (Chord)
- Hierarchical (MBone)
- Hybrid (EDonkey)





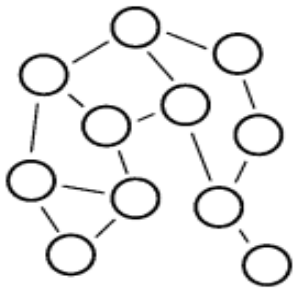






Unstructured P2P Networks

- Gnutella, JxTA, FreeNet
- Techniques:
  - Flooding
  - Replication & Caching
  - Time To Live (TTL)
  - Epidemics & Gossiping protocols
  - Super-Peers
  - Random Walkers & Probabilistic algorithms



Gnutella Protocol v0.4

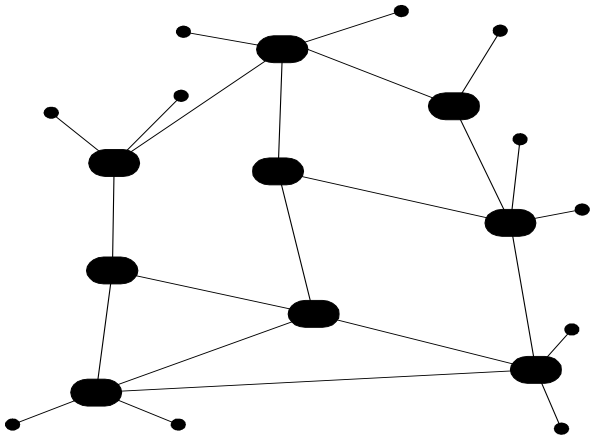
- One of the most popular **file-sharing** protocols.
- Operates without a central Index Server (such as Napster).
- Clients (downloaders) are also servers => ***servents***
- Clients may join or leave the network at any time => highly **fault-tolerant** but with a cost!
- Searches are done within the virtual network while actual downloads are done offline (with HTTP).
- The core of the protocol consists of 5 **descriptors** *(PING, PONG, QUERY, QUERHIT and PUSH).*

Gnutella Protocol

- It is important to understand how the protocol works in order to understand our framework.
- A *Peer (p)* needs to connect to 1 or more other Gnutella Peers in order to participate in the virtual Network
- *p* initially *doesn't know IPs of its fellow file-sharers*

•Servent *p*

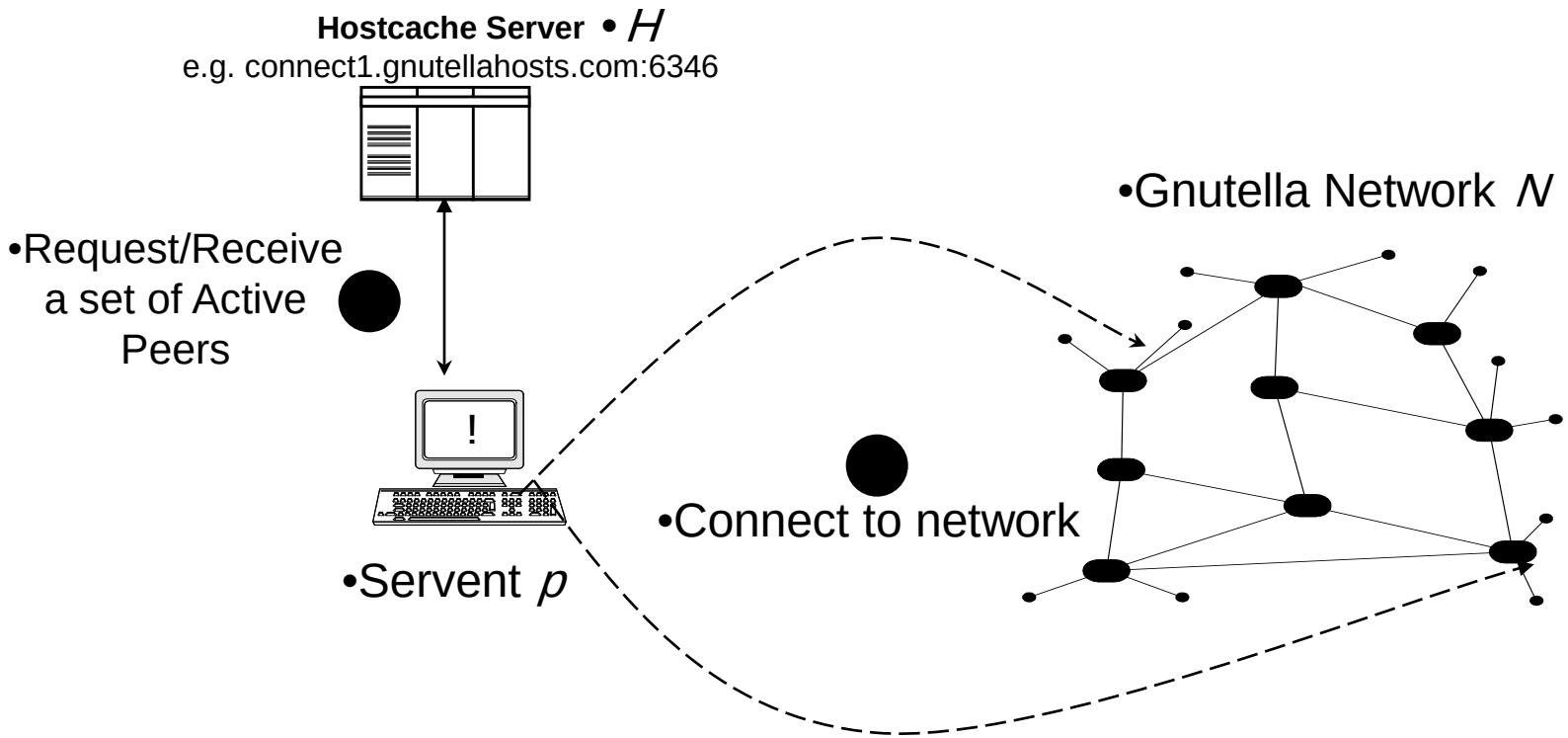
•Gnutella Network *N*



Gnutella Protocol

a. HostCaches – The initial connection

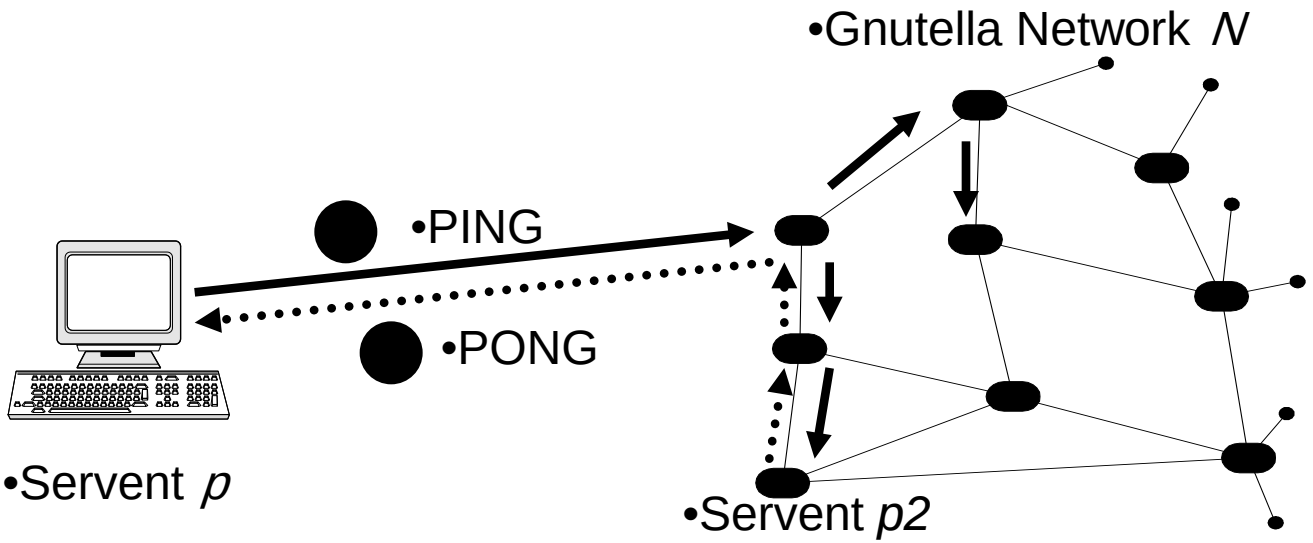
- ***P*** connects to a HostCache H to obtain a set of IP addresses of active peers.
- ***P*** might alternatively probe its cache to find peers it was connected in the past.



Gnutella Protocol

b. Ping/Pong – The communication overhead

- Although *p* is already connected it must discover new peers since its current connections may break.
- Thus, it sends periodically PING messages which are broadcasted (message flooding).
- If a host e.g. *p2* is available it will respond with a PONG (routed only the same path the PING came from).
- *P* might utilize this response and attempt a connection to p2 in order to increase its degree.



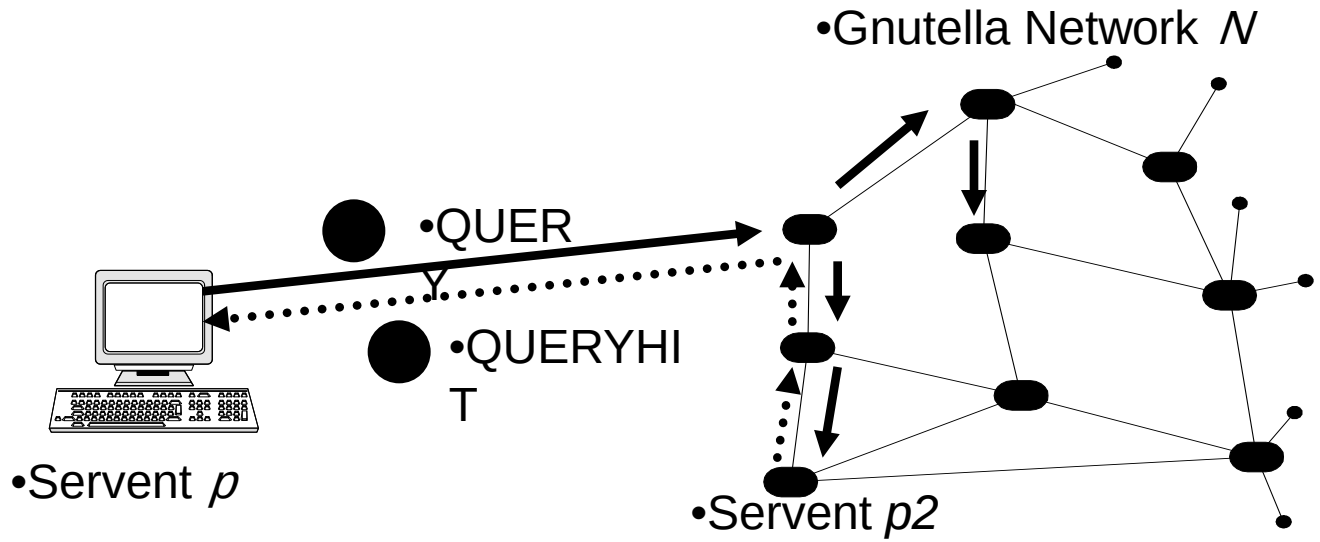
Gnutella Protocol

c. Query/QueryHit – The utilization

- Query descriptors contain unstructured queries e.g. “celine dion mp3”
- They are again, like PING, broadcasted with a typical **TTL**=7.
- If a host e.g. *p2* matches the query it will respond with a Queryhit descriptor

d. Push – Enable downloads from peers that are firewalled.

- If a peer is firewalled => we can’t connect to him. Hence we request from him to establish a connection on us and to send us the file.



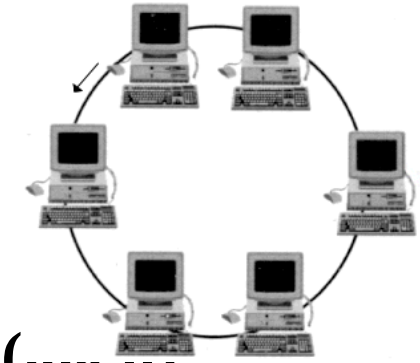
Search Performance of Gnutella

- Breadth-first search always finds the optimal path
- Performance is the same under random and target failure
- Scales logarithmically in search path length
- The search bandwidth used by query increases proportionally with the number of the nodes in the network.

Gnutella Conclusions

- **The Gnutella communication overhead is huge.**
  Ping/Pong: 63% | Query/QueryHits: 37%.
- **Gnutella users can be classified in three main categories.**
  Season-Content, Adult-Content and File Extension Searchers.
- **Gnutella Users are mainly interested in *video* > *audio* > *images* > *documents.***
- Although Gnutella is a truly international phenomenon **its largest segment is contributed by only a few countries.**
- The clients started **conforming** to the specifications of the protocol and that they **thwart excessive network resources consumption**.
- **Free riding**: “Specifically, we found that nearly 70% of Gnutella users share no files, and nearly 50% of all responses are returned by the top 1% of sharing hosts”.

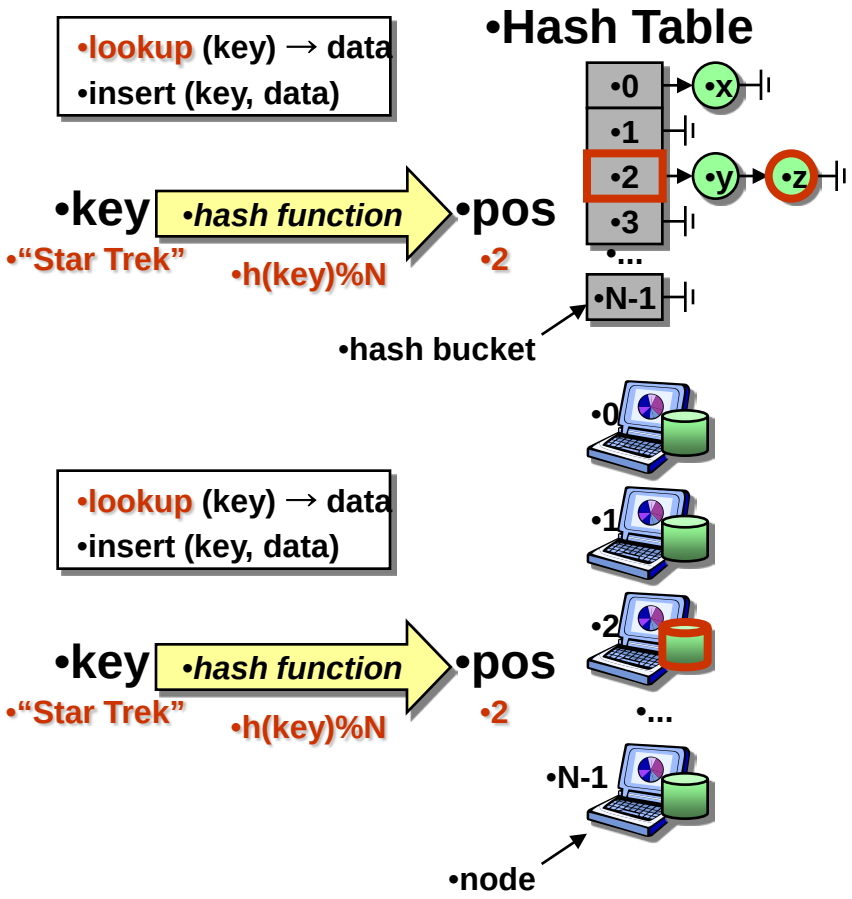
Structured P2P Networks



- Nodes form a regular topology:
  - Chord (Ring)
  - Kademlia (Tree)
  - CAN (Hypercube)
- Assure routing performance to locate contents **O (log n)**
- Every node has a unique id
  - Distance metric over Ids
- Every node has a routing table with links to other nodes
- They are also called Distributed Hash Tables (DHTs)

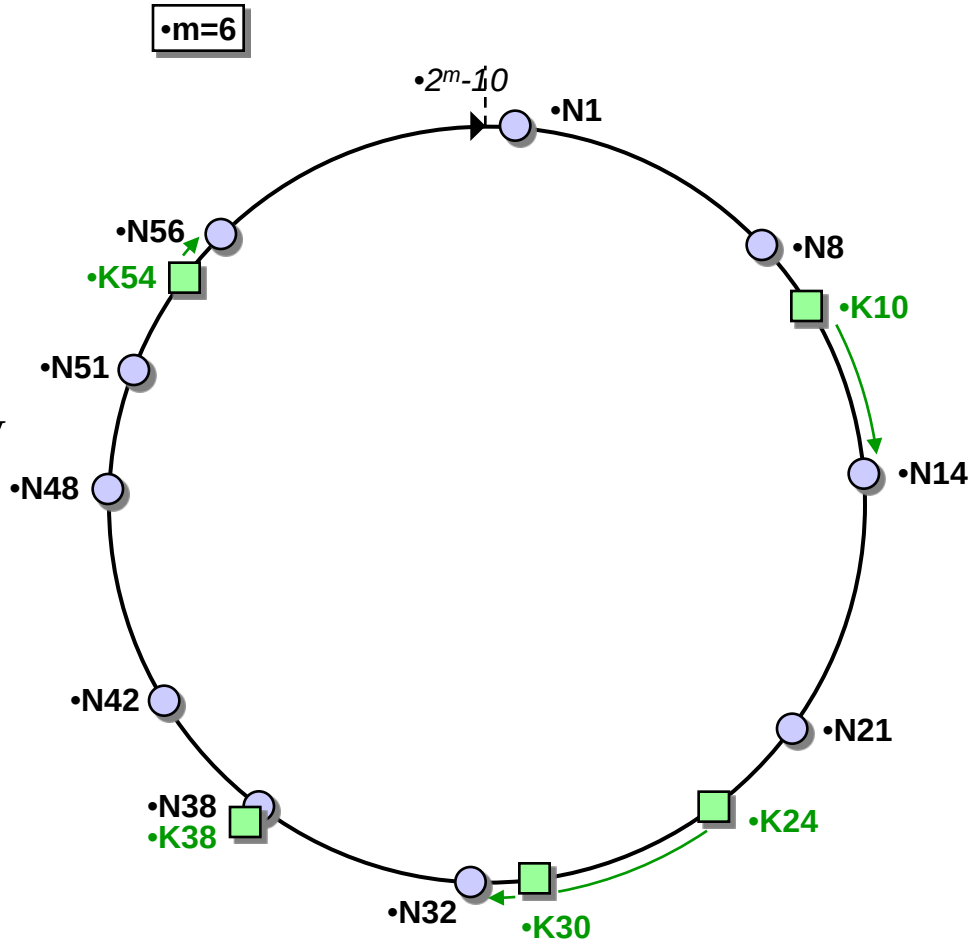
Distributed Hash Tables

- A hash table links data with keys
  - The key is hashed to find its associated *bucket* in the hash table
  - Each *bucket* expects to have #elems/#buckets elements
- In a **Distributed Hash Table (DHT)**, physical nodes are the hash *buckets*
  - The key is hashed to find its responsible node
  - Data and load are balanced among nodes



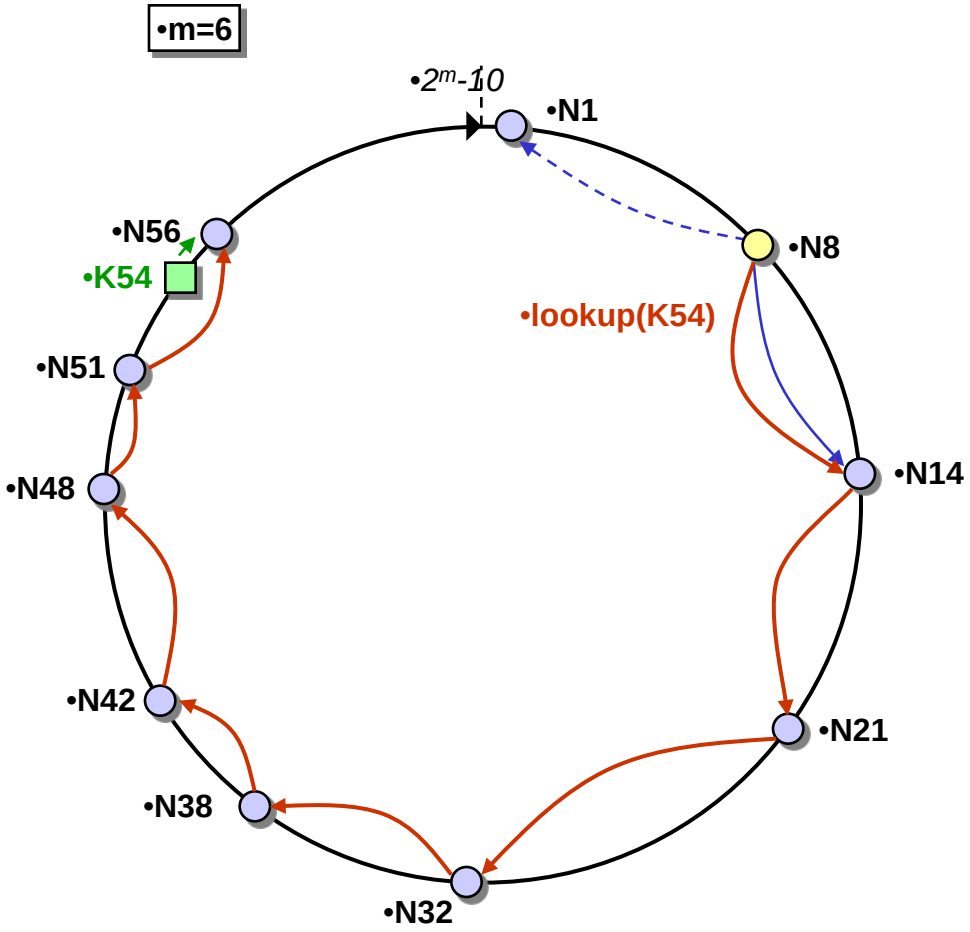
Chord (MIT)

- Circular *m*-bit ID space for both keys and nodes
- Node ID = SHA-1(IP address)
- Key ID = SHA-1(key)
- A key is mapped to the first node whose ID is equal to or follows the key ID
  - Each node is responsible for *O(K/N)* keys
  - *O(K/N)* keys move when a node joins or leaves



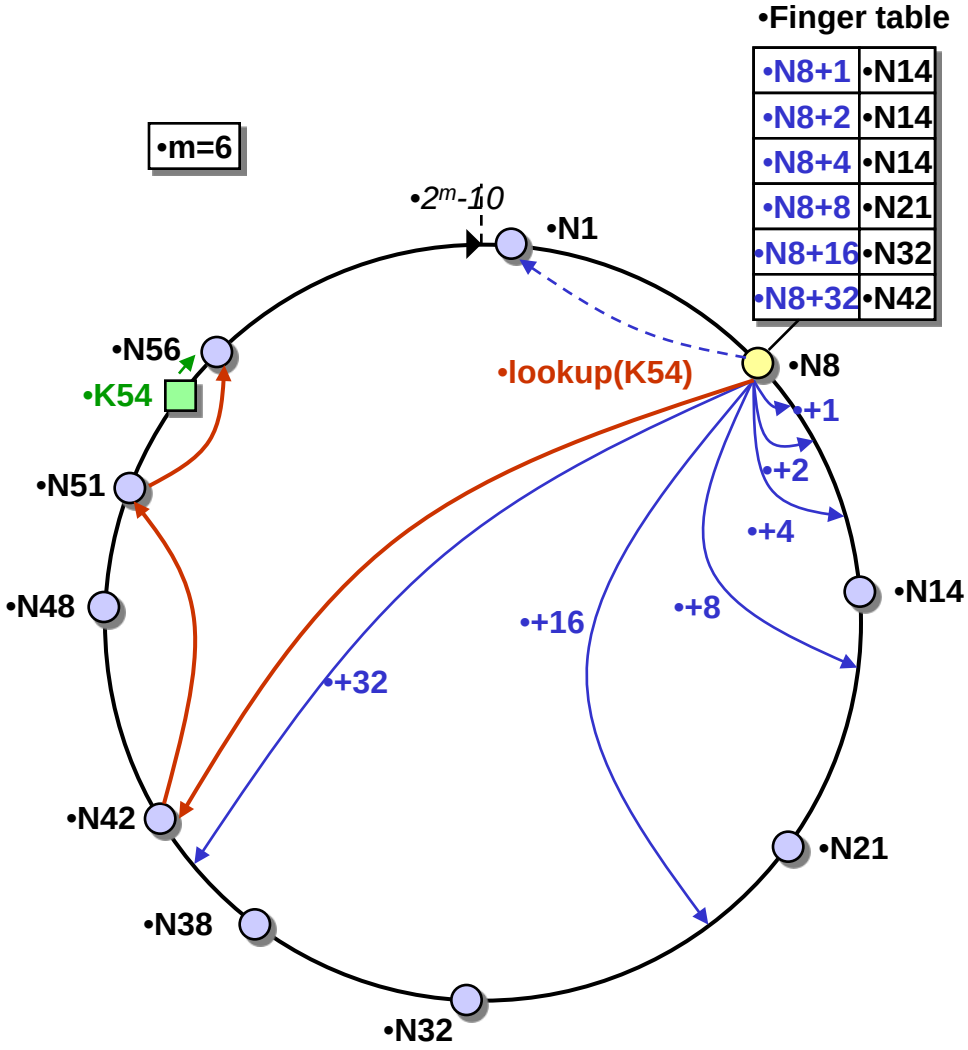
Chord State and Lookup (1)

- Basic Chord: each node knows only 2 other nodes on the ring
  - Successor
  - Predecessor (for ring management)
- Lookup is achieved by forwarding requests around the ring through successor pointers
  - Requires *O(N)* hops



Chord State and Lookup (2)

- Each node knows *m* other nodes on the ring
  - Successors: finger *i* of *n* points to node at $n+2^i$ (or successor)
  - Predecessor (for ring management)
  - *O(log N)* state per node
- Lookup is achieved by following closest preceding fingers, then successor
  - *O(log N)* hops



Chord Ring Management

- For correctness, Chord needs to maintain the following invariants
  - For every key *k*, *succ(k)* is responsible for *k*
  - Successor pointers are correctly maintained
- Finger table are not necessary for correctness
  - One can always default to successor-based lookup
  - Finger table can be updated lazily

Joining the Ring

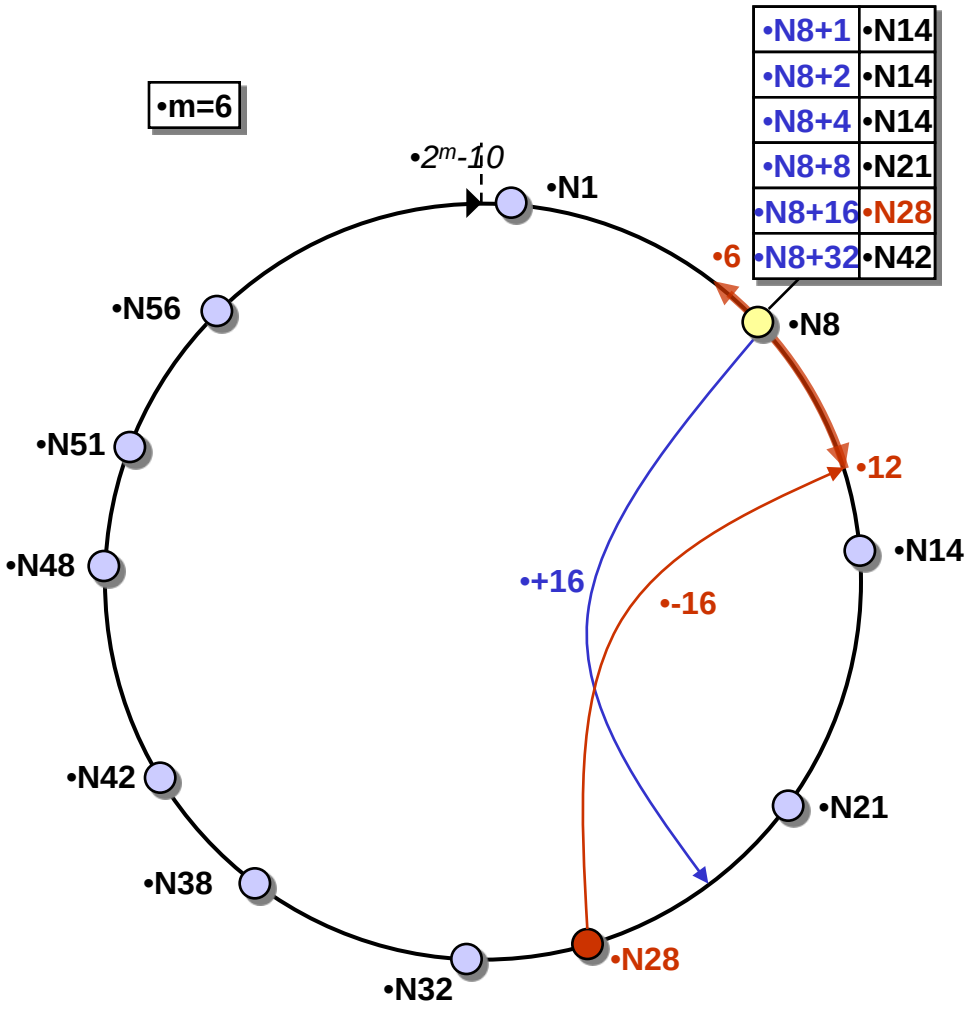
- Three step process:
  - Initialize all fingers of new node
  - Update fingers of existing nodes
  - Transfer keys from successor to new node

Joining the Ring — Step 1

- Initialize the new node finger table
  - Locate any node *n* in the ring
  - Ask *n* to lookup the peers at $j+2^0$, $j+2^1$, $j+2^2$…
  - Use results to populate finger table of *j*

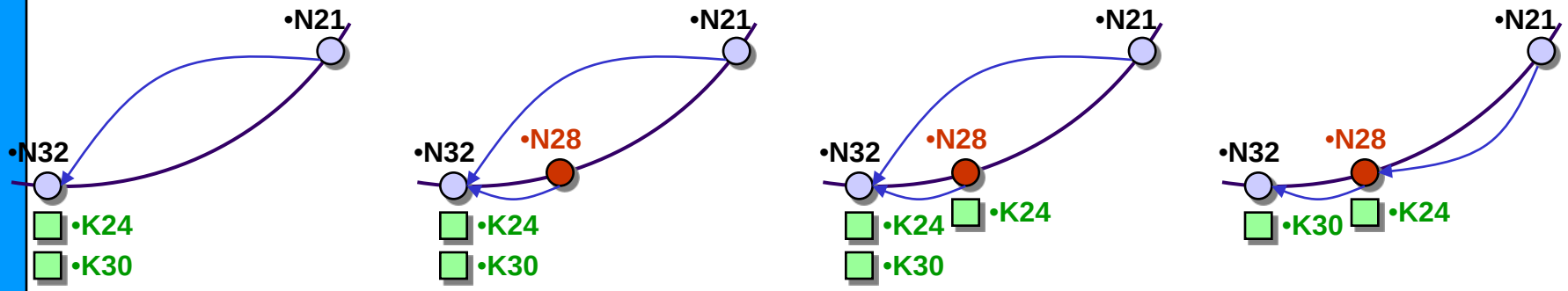
Joining the Ring — Step 2

- Updating fingers of existing nodes
  - New node *j* calls update function on existing nodes that must point to *j*
    - Nodes in the ranges $[j\text{-}2^i , pred(j)\text{-}2^i+1]$
  - *O(log N)* nodes need to be updated



Joining the Ring — Step 3

- Transfer key responsibility
  - Connect to successor
  - Copy keys from successor to new node
  - Update successor pointer and remove keys
- Only keys in the range are transferred

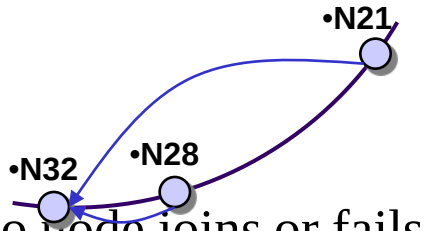


Stabilization

- Case 1: finger tables are reasonably fresh
- Case 2: successor pointers are correct, not fingers
- Case 3: successor pointers are inaccurate or key migration is incomplete — MUST BE AVOIDED!
- Stabilization algorithm periodically verifies and refreshes node pointers (including fingers)
  - Basic principle (at node *n*):

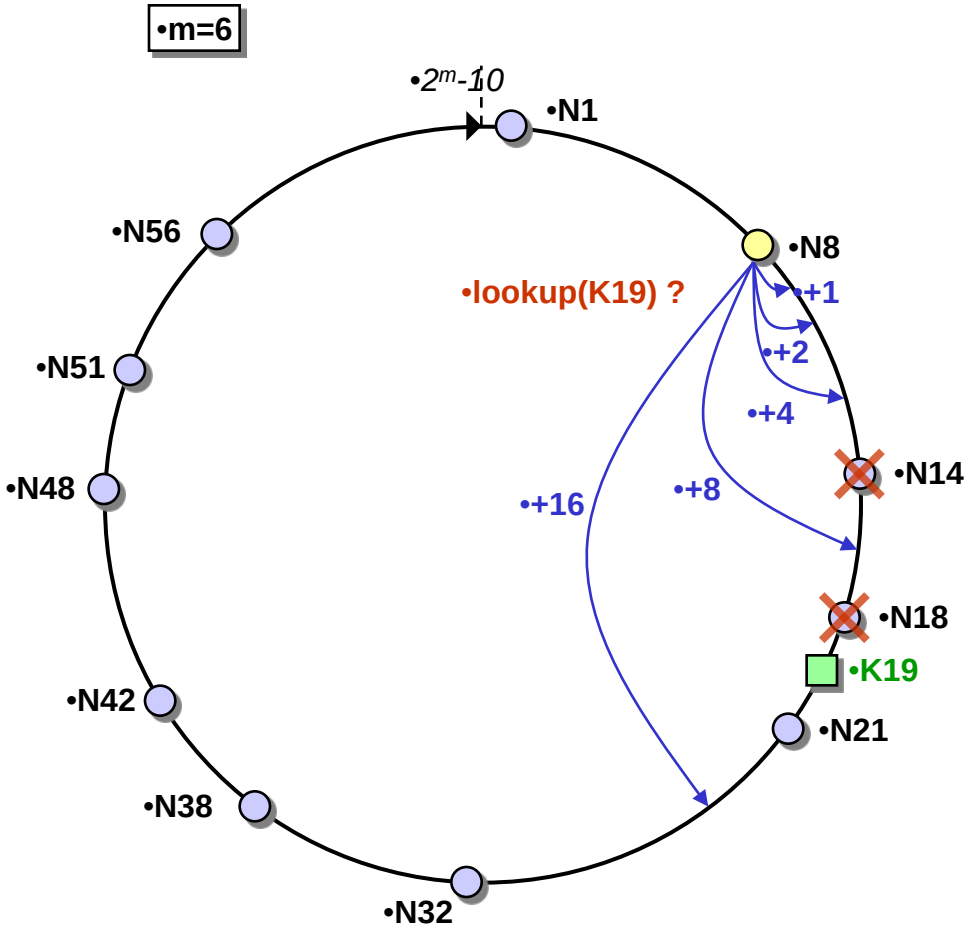
    x = *n.succ.pred*
    if *x* ∈ (*n*, *n.succ*)
      n = *n.succ*
      notify *n.succ*

  - Eventually stabilizes the system when no node joins or fails

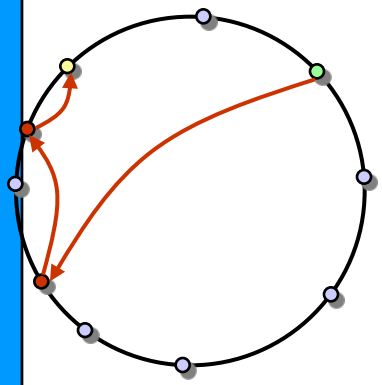


Dealing With Failures

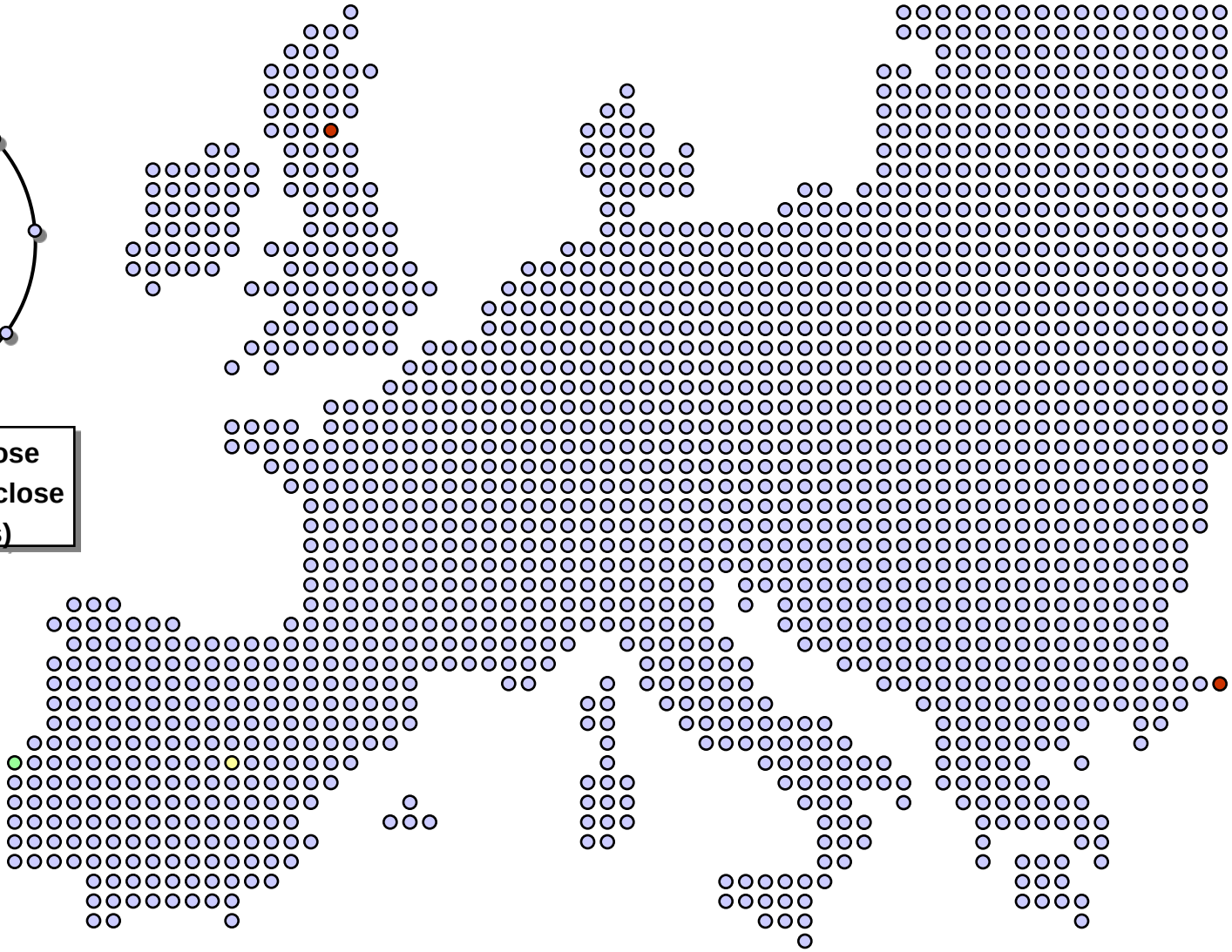
- Failure of nodes might cause incorrect lookup
  - N8 doesn't know correct successor, so lookup of K19 fails
- Solution: successor list
  - Each node *n* knows *r* immediate successors
  - After failure, *n* knows first live successor and updates successor list
  - Correct successors guarantee correct lookups



Chord and Network Topology



- Nodes numerically-close
- are not topologically-close
- (1M nodes = 10+ hops)



Kademlia

One major goal of P2P systems is file or object lookup: Given a data item X stored at some set of nodes in the system, find it.

Unlike Chord, CAN, or Pastry **Kademlia uses Tree-based routing**

Kademlia

- Nodes, files and key words, deploy SHA-1 hash into a 160 bits space.
- Every node maintains information about files, key words **close to itself**.
- The closeness between two objects measured as their bitwise XOR interpreted as an integer.

  distance(a, b) = a XOR b

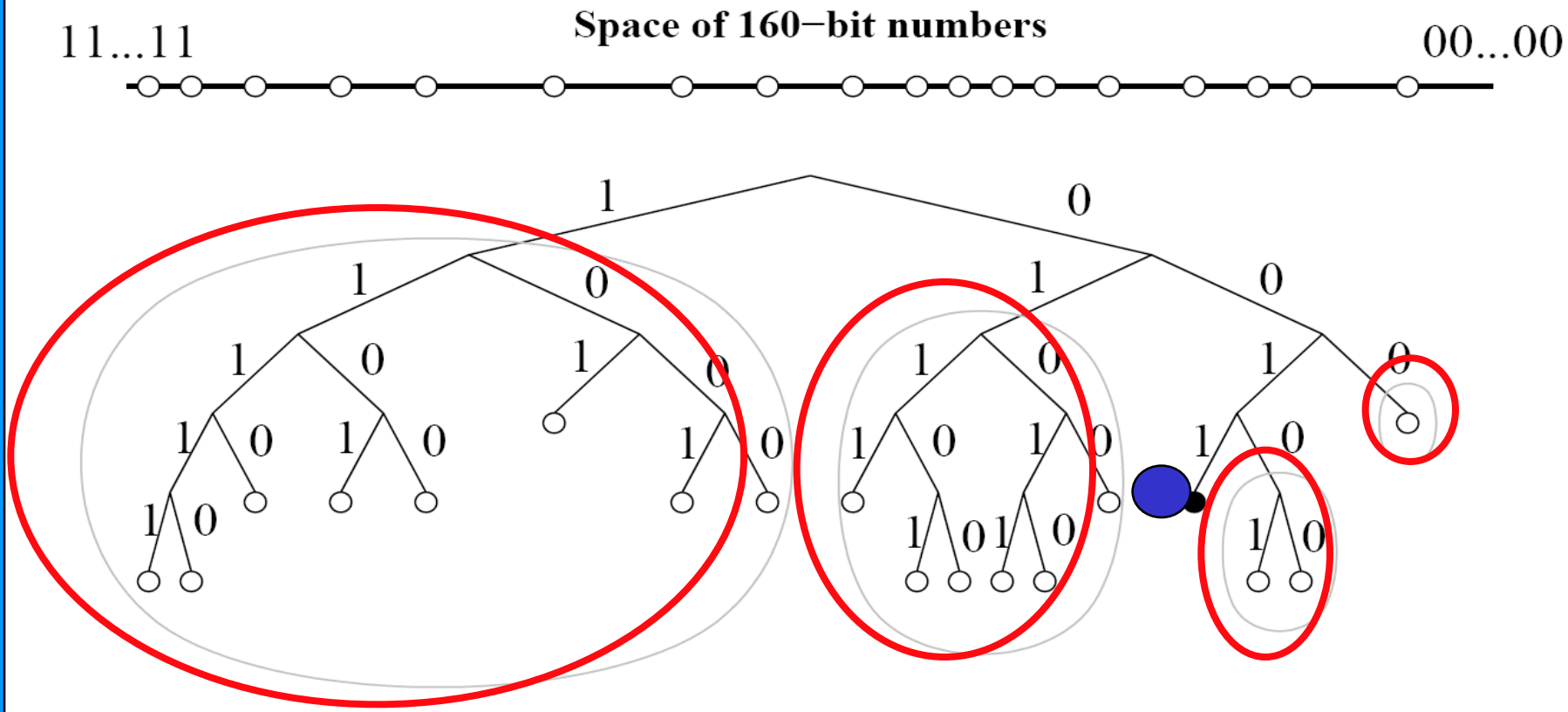
Claims

- Only a small number of configuration messages sent by the nodes.
- Uses *parallel asynchronous queries* to avoid timeout delays of the failed nodes. *Routes are selected based on latency*
- Unlike (unidirectional) Chord Kademlia is symmetric i.e. **dist (a,b) = dist (b,a)**

Kademlia Binary Tree

- Treat nodes as leaves in a binary tree.
- Start from root, for any given node, dividing the binary tree into a series of successively lower subtrees that don’t contain the node.

Kademlia Binary Tree

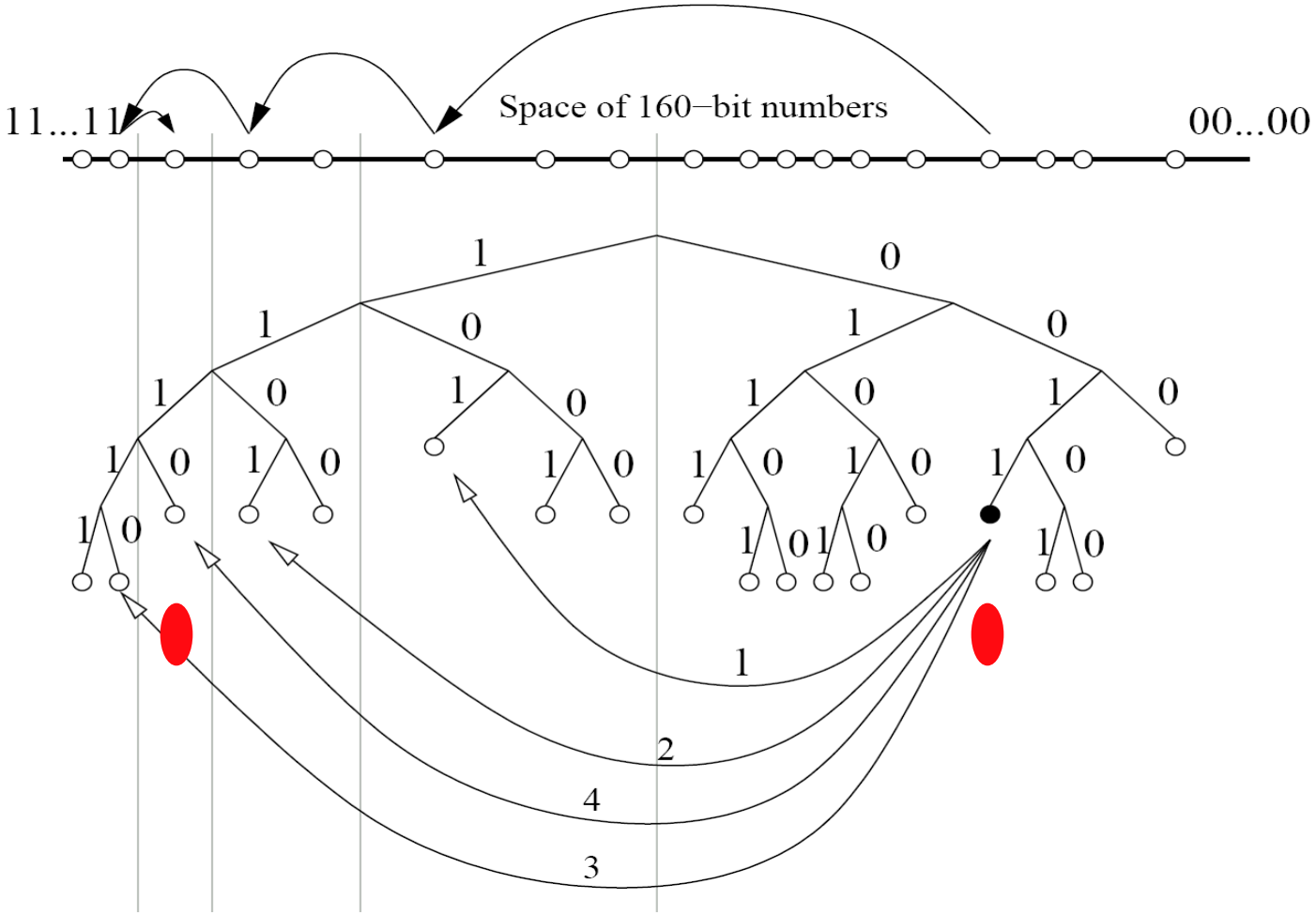


Subtrees of interest for a node 0011……

Kademlia Binary Tree

- Every node keeps touch with **at least one node from each of its subtrees**. (if there is a node in that subtree.) Corresponding to each subtree, there is a **k-bucket**.
- Every node keeps a list of (IP, Port, Node id) triples, and (key, value) tuples for further exchanging information with others.

Kademlia Search



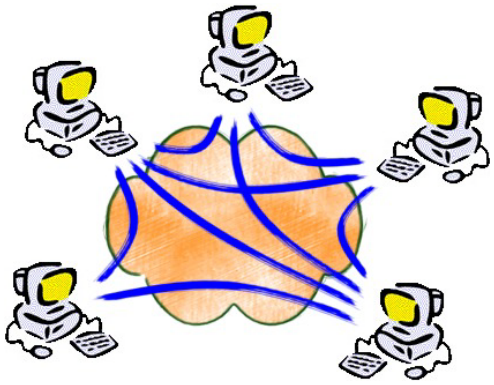
•An example of lookup: node 0011 is searching for 1110……in the network

Summary (1/1)

- With its novel XOR-based metric topology, Kademlia is the first peer-to-peer system to combine provable consistency and performance, latency-minimizing routing, and a symmetric, unidirectional topology.
- Kademlia is the first peer-to-peer system to exploit the fact that node failures are inversely related to uptime.

DHT applications

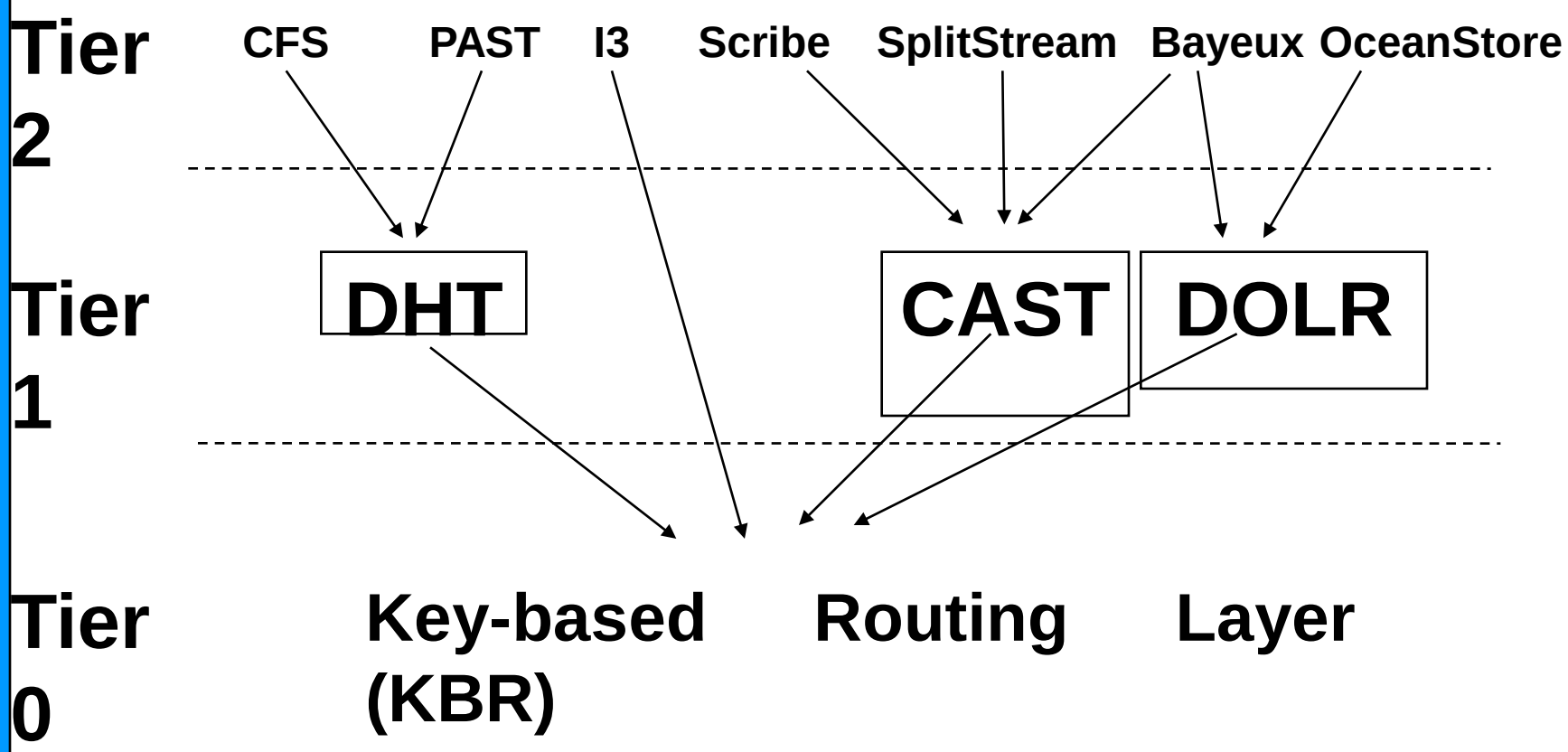
- File sharing [CFS, OceanStore, PAST, …]
- Web cache [Squirrel, Coral]
- Censor-resistant stores [Eternity, FreeNet, …]
- Application-layer multicast [Narada, …]
- Event notification [Scribe]
- Streaming (SplitStream)
- Naming systems [ChordDNS, INS, …]
- Query and indexing [Kademlia, …]
- Communication primitives [I3, …]
- Backup store [HiveNet]
- Web archive [Herodotus]
- Suggestions ?



Common API Proposal

- Easily supportable by old and new protocols
- Enables application portability between protocols
- Enables common benchmarks
- Provides a framework for reusable components
- Achieves Interoperability between heterogeneous protocols

A Layered Approach: Common API



Functional Layers

- DHT (Distributed Hash Table)
  - **put**(key,data), value = **get**(key)
  - Hashtable layered across network
  - Handles replication; distributes replicas randomly
  - Routes queries towards replicas by name
- DOLR (Decentralized Object Location and Routing)
  - **publish**(objectId), **route**(msg, nodeId),
    r**outeObj**(msg, objectId, n)
  - Application controls replication and placement
  - Cache location pointers to replicas; queries quickly intersect pointers and redirect to nearby replica(s)

API (Routing)

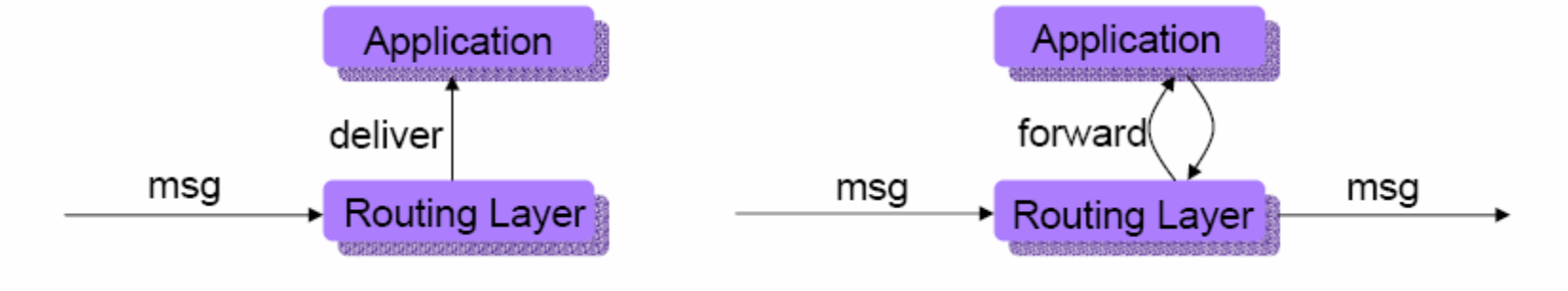
- Data types
  - Key, nodeId = 160 bit integer
  - Node = Address (IP + port #), nodeId
  - Msg: application-specific msg of arbitrary size

- Invoking routing functionality
  - **Route**(key, msg, [node])
    - route message to node currently responsible for key
    - Non-blocking, best effort – message may be lost or duplicated.
    - node: transport address of the node last associated with key (proposed first hop, optional)

API

- **nextHopSet = local_lookup(key, num, safe)**
  - Returns a set of at most *num* nodes from the local routing table that are possible next hops towards the *key*.
  - Safe: whether choice of nodes is randomly chosen
- **nodehandle[ ] = neighborSet(max_rank)**
  - Returns unordered set of nodes as neighbors of the current node.
  - Neighbor of rank *i* is responsible for keys on this node should all neighbors of rank < *i* fail
- **nodehandle[ ] = replicaSet(key, num)**
  - Returns ordered set of up to *num* nodes on which replicas of the object with key *key* can be stored.
  - Result is subset of neighborSet plus local node
- **boolean = range(node, rank, lkey, rkey)**
  - Returns whether current node would be responsible for the range specified by *lkey* and *rkey*, should the previous *rank-1* nodes fail.

API (Upcalls)



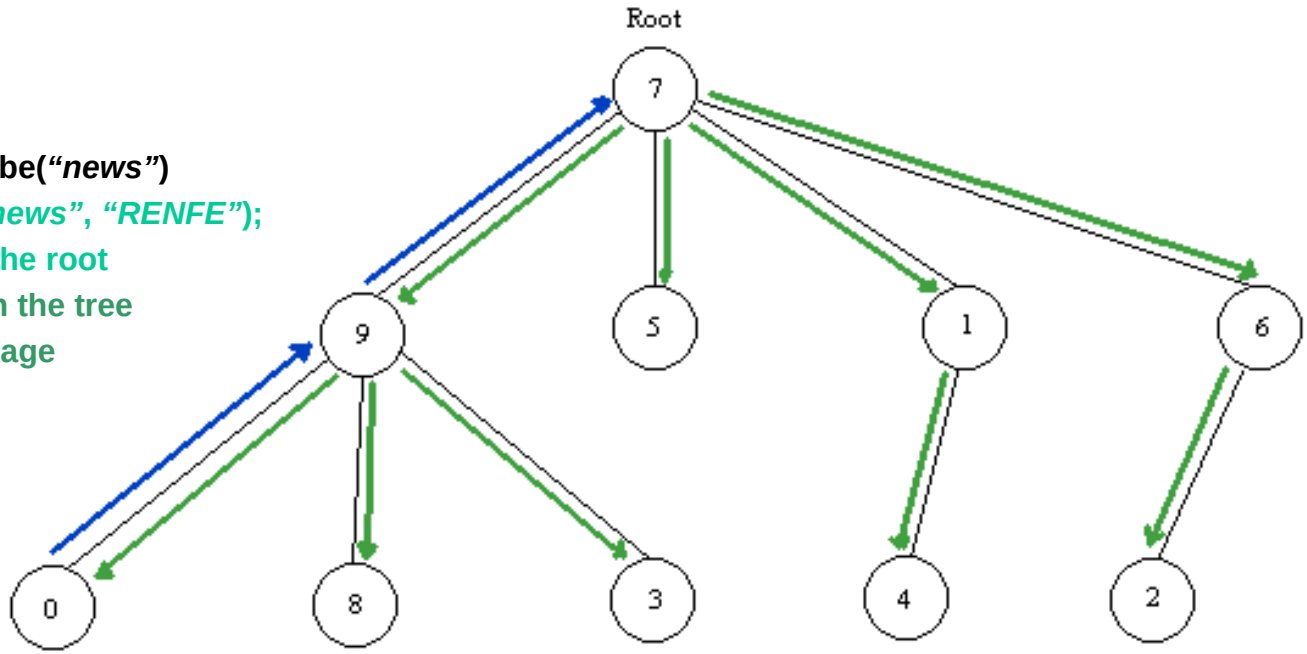
- **Deliver(key, msg)**
  - Delivers an incoming message to the application. One application per node.
- **Forward(key, msg, nextHopNode)**
  - Synchronous upcall invoked at each node along route
  - On return, will forward *msg* to *nextHopNode*
  - App may modify *key*, *msg*, *nextHopNode*, or terminate by setting *nextHopNode* to NULL.
- **Update(node, boolean joined)**
  - Upcall invoked to inform app of a change in the local node's neighborSet, either a new node joining or an old node leaving.

CAST: Scribe

- Application Level Multicast using FreePastry
- ANYCAST, MANYCAST, MULTICAST
- 1 to Many communication
- API
  - Create (credentials, topicId).
  - Subscribe(credentials, topicId, eventHandler).
  - Unsubscribe (credentials, topicId).
  - Publish (credentials, topicId, event).

Example :

- **All nodes -> subscribe(*"news"*)**
- **Node 0 -> publish(*"news"*, *"RENFE"*);**
- **Message arrives to the root**
- **And then goes down the tree**
- **All receive the message**



Common API ideas

- It is a paper, not a standard !
  - Towards a Common API for Structured Peer-to-Peer Overlays. IPTPS 2003.
  - Dabek, Zhao, Druschel, Stoica, kubiatowitcz
- It is academic !! There are not comercial implementations of the Common API
- But: it helps to provide common concepts for developing p2p applications over structured overlays

Common API = KBR

- You still have to implement the DHT, replication protocols, fault tolerance, …
- Other approach is to construct directly applications on top of the DHT (put, get) interface. Example: OpenDHT has public XMLRPC APIs.

Understanding BitTorrent

By Iqbal Mohomed

What is the Problem Being Solved Here?

- Sharing a fairly large file
- Involves making a replica
- Problem is somewhat similar to, but not the same as, replication in a distributed file system, a Content Delivery Network or a Distributed Hash Table overlay network

Simple Solution: One Big Server

- Make the file available on a central server
- Each client downloads file from this server
- Problems
  - Solution does not scale very well
  - With a large number of clients, the server's resources get overwhelmed

The Brilliance of Napster: P2P

- In the original Napster, nodes connected to a central server and gave it a listing of all the files they had.
- Nodes relay searches to the central server, which performs them locally
- The actual file transfer occurs peer-to-peer
- The big weakness of this approach was that the directory server was a single point of failure

The Gnutella Solution

- In Gnutella, all nodes are true peers
- This gets rid of the single point of failure problem
- The new problem is efficiency and scalability
  - Specifically, searches go across a large number of nodes, generating a massive amount of traffic
  - There is a larger compromise in privacy as peers see search queries

The FastTrack Network

- aka Kazaa
- Combines the Napster and Gnutella approaches
- Nodes connect to super-peers that act as the directory server in Napster
- The super-peers connect to each other similar to Gnutella
- This solution is working very well in practice

Enter BitTorrent

- Released in the summer of 2001
- Uses basic ideas from game theory to largely eliminate the free-rider problem
  - All previous systems could not deal with this problem well
- Makes no strong guarantees unlike DHTs
- It is working extremely well in practice, unlike DHTs ☺

Basic Idea

- Chop file into many pieces
- Replicate DIFFERENT pieces on different peers as soon as possible
- As soon as a peer has a complete piece, it can trade it with other peers
- Hopefully, we will be able to assemble the entire file at the end

Basic Components

- Seed
  - Peer that has the entire file
- Leacher
  - Peer that has an incomplete copy of the file
- A Torrent file
  - Passive component
  - Files are typically fragmented into 256KB pieces
  - The torrent file lists SHA1 hashes of all the pieces to allow peers to verify integrity
  - Typically hosted on a web server
- A Tracker
  - Active component
  - Allows peers to find each other
  - Returns a random list of peers

Piece Selection

- The order in which pieces are selected by different peers is critical for good performance
- If a bad algorithm is used, we could end up in a situation where every peer has all the pieces that are currently available and none of the missing ones
- If the original seed is taken down, the file cannot be completely downloaded!

Rarest Piece First

- Policy: Determine the pieces that are most rare among your peers and download those first
- This ensures that the most common pieces are left till the end to download
- Rarest first also ensures that a large variety of pieces are downloaded from the seed

Choking

- One of BitTorrent's most powerful idea is the choking mechanism
- It ensures that nodes cooperate and eliminates the free-rider problem
- Cooperation involves uploaded sub-pieces that you have to your peer
- Choking is a temporary refusal to upload; downloading occurs as normal
- Connection is kept open so that setup costs are not borne again and again
- Based on game-theoretic concepts
  - Tit-for-tat strategy in Repeated Games

Prisoner's Dilemma

	Cooperate	Defect
Cooperate	3, 3	0, 5
Defect	5, 0	1, 1

	Cooperate	Defect
Cooperate	win-win	lose much-win much
Defect	win much-lose much	lose-lose

Choking Algorithm

- Goal is to have several bidirectional connections running continuously
- Upload to peers who have uploaded to you recently
- Unutilized connections are uploaded to on a trial basis to see if better transfer rates could be found using them

Choking Specifics

- A peer always unchokes a fixed number of its peers (default of 4)
- Decision to choke/unchoke done based on current download rates, which is evaluated on a rolling 20-second average
- Evaluation on who to choke/unchoke is performed every 10 seconds
  - This prevents wastage of resources by rapidly choking/unchoking peers
  - Supposedly enough for TCP to ramp up transfers to their full capacity
- Which peer is the optimistic unchoke is rotated every 30 seconds

Upload-Only mode

- Once download is complete, a peer has no download rates to use for comparison nor has any need to use them
- The question is, which nodes to upload to?
- Policy: Upload to those with the best upload rate.
- This ensures that pieces get replicated faster
- Also, peers that have good upload rates are probably not being served by others

Trackerless torrents

BitTorrent also supports "trackerless" torrents, featuring a DHT implementation that allows the client to download torrents that have been created without using a BitTorrent tracker.

References

- "BitTorrent Economics Paper" , Bram Cohen
- "BitTorrent protocol specification" , Bram Cohen
- "BitTorrent Resource Availability Analysis" , Brian Greinke and James Hsia. (Rice)
- "Dissecting BitTorrent: Five Months in a Torrent's Lifetime" , M. Izal, G. Urvoy-Keller, E.W. Biersack, P.A. Felber, A. Al Hamra, and L. Garc es-Erice. (Institut Eurecom, France)